SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Yellampati Swathi	**Reg. No.:**	192324095
Section / Batch:	AIDS-2023	**Date:**	27/10/2026

Code of Conduct

I Yellampati swathi(192324095) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: y.swathi

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. **Problem Statement:**

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address
Password
Mobile Number

using Regular Expressions.

Validation Rules

Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'. Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters
At least one uppercase letter
At least one lowercase letter
At least one digit
At least one special character (@,#,$,%,&,!)

Mobile Number

Must contain exactly 10 digits. First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

- **Problem Statement**

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
  - States
  - Input alphabet
  - Transition table
  - Initial state
  - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:
Strings ending with "ab"

Input
abaab

Output
Transition Path:
q0 → q1 → q0 → q1 → q2
Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. **Problem Statement**

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:
Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu

- Search Date
- Search Phone Number
- Search Hashtag
- Search Mention
- Search Prefix
- Search Suffix

Expected Output
Display all matching patterns based on user selection.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Section 1: Intelligent Email and Password Validator

1.1 Approach

Each credential is checked against one regular expression using re.fullmatch, so the whole string must satisfy the rule. The password rule uses positive lookaheads, which test each requirement independently without consuming characters.

Field	Regular Expression	Meaning
Email	^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$	Starts with a letter; letters, digits, '.', '_' before @; alphabetic domain; allowed extensions
Password	^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$	Uppercase, lowercase, digit and special character lookaheads; at least 8 characters
Mobile	^[6-9]\d{9}$	First digit 6 to 9 followed by exactly 9 more digits

1.2 Source Code (validator.py)

```
import re

EMAIL_PATTERN = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$"
PASSWORD_PATTERN = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$"
MOBILE_PATTERN = r"^[6-9]\d{9}$"


def validate_email(email):
    return bool(re.fullmatch(EMAIL_PATTERN, email))


def validate_password(password):
    return bool(re.fullmatch(PASSWORD_PATTERN, password))


def validate_mobile(mobile):
    return bool(re.fullmatch(MOBILE_PATTERN, mobile))


def main():
    print("=== User Registration Validator ===")
    email = input("Enter Email    : ").strip()
    password = input("Enter Password : ").strip()
    mobile = input("Enter Mobile   : ").strip()
    print()
    print("Valid Email" if validate_email(email) else "Invalid Email")
    print("Strong Password" if validate_password(password) else "Weak Password")
    print("Valid Mobile Number" if validate_mobile(mobile) else "Invalid Mobile Number")


if __name__ == "__main__":
    main()
```

1.3 Output

```python
import re

email = input("Enter Email Address: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

email_pattern = r"[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)"
password_pattern = r"(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}"
mobile_pattern = r"[6-9]\d{9}"

if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")

if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")

if re.fullmatch(mobile_pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

```
Enter Email Address: swathiyellampati636@gmail.com
Enter Password: swa@123
Enter Mobile Number: 6281093795
Valid Email
Weak Password
Valid Mobile Number
```

Figure 1: Email, password and mobile number validation output

Section 2: DFA Simulator

2.1 Approach

The DFA is stored as a five-part description: states, input alphabet, transition table (a dictionary keyed by (state, symbol)), initial state and set of final states. For every input string the simulator starts at the initial state, follows one transition per symbol, records the visited states and accepts if the last state is final. A symbol outside the alphabet causes rejection.

Language used in the demonstration: strings over {a, b} ending with "ab".

State	on a	on b
q0 (start)	q1	q0
q1	q1	q2
q2 (final)	q1	q0

2.2 Source Code (dfa.py)

```
def read_dfa():
    """Read the DFA description from the user."""
    states = input("Enter states (space separated)        : ").split()
    alphabet = input("Enter input alphabet (space separated): ").split()
    print("Enter transitions as: <state> <symbol> <next_state>")
    transitions = {}
    for _ in range(len(states) * len(alphabet)):
        s, a, t = input("  transition: ").split()
        transitions[(s, a)] = t
    start = input("Enter initial state                  : ").strip()
    finals = set(input("Enter final state(s) (space sep.)    : ").split())
    return states, alphabet, transitions, start, finals



def simulate(dfa, string):
    states, alphabet, transitions, start, finals = dfa
    state = start
    path = [state]
    for ch in string:
        if ch not in alphabet or (state, ch) not in transitions:
            return path, False          # invalid symbol -> reject
        state = transitions[(state, ch)]
        path.append(state)
    return path, state in finals



def main():
    print("=== DFA Simulator ===")
    dfa = read_dfa()
    print("\nEnter input strings (type 'exit' to stop)")
    while True:
        s = input("\nInput: ").strip()
        if s.lower() == "exit":
            break
        path, ok = simulate(dfa, s)
        print("Transition Path:")
        print(" → ".join(path))
        print("Accepted" if ok else "Rejected")



if __name__ == "__main__":
```

main()

2.3 Output

```
states = ["q0", "q1", "q2"]
alphabet = ["a", "b"]
initial_state = "q0"
final_states = ["q2"]

transition = {
    "q0": {"a": "q1", "b": "q0"},
    "q1": {"a": "q1", "b": "q2"},
    "q2": {"a": "q1", "b": "q0"}
}

n = int(input("How many strings? "))

for i in range(n):
    text = input("Enter string (a/b): ")
    state = initial_state
    path = [state]
    valid = True

    for ch in text:
        if ch not in alphabet:
            valid = False
            break

        state = transition[state][ch]
        path.append(state)

    print("Transition Path:", " -> ".join(path))

    if valid and state in final_states:
        print("Accepted")
    else:
        print("Rejected")
```

```
How many strings? 3
Enter string (a/b): a
Transition Path: q0 -> q1
Rejected
Enter string (a/b): a
Transition Path: q0 -> q1
Rejected
Enter string (a/b): b
Transition Path: q0 -> q0
Rejected
```

Figure 2: DFA simulator output for the language ending with "ab"

Note: for the input abaab the correct path has six states because there are five symbols: q0 → q1 → q2 → q1 → q1 → q2. The path shown in the problem statement example has only five states, so it cannot be the path for a five-symbol string. The result, Accepted, is the same.

Section 3: Smart Pattern Matching Engine

3.1 Approach

The program reads multi-line text and shows a menu. Each option applies one regular expression with re.findall and prints every match.

Search	Regular Expression	Matches
Date	\b\d{1,2}[/-]\d{1,2}[/-]\d{4}\b	12/09/2026
Phone number	\b[6-9]\d{9}\b	9876543210
Hashtag	#\w+	#NLP
Mention	@\w+	@OpenAI
Prefix	\b<prefix>\w*	Words starting with the prefix
Suffix	\b\w*<suffix>\b	Words ending with the suffix
Word	\b<word>\b	Whole-word match

User input for prefix, suffix and word is passed through re.escape so special characters are treated literally. Options 1 to 6 follow the required menu; option 7 (Word Search) and option 8 (Exit) were added to cover the Word Search requirement.

3.2 Source Code (search_engine.py)

```
import re

PATTERNS = {
   "date":    r"\b\d{1,2}[/-]\d{1,2}[/-]\d{4}\b",
   "phone":   r"\b[6-9]\d{9}\b",
   "hashtag": r"#\w+",
   "mention": r"@\w+",
}


def search(pattern, text):
   return re.findall(pattern, text)


def word_search(word, text):
   return search(r"\b" + re.escape(word) + r"\b", text)


def prefix_search(prefix, text):
   return search(r"\b" + re.escape(prefix) + r"\w*", text)


def suffix_search(suffix, text):
   return search(r"\b\w*" + re.escape(suffix) + r"\b", text)


def read_text():
   print("Enter text (blank line to finish):")
   lines = []
   while True:
      line = input()
      if not line:
         break
```

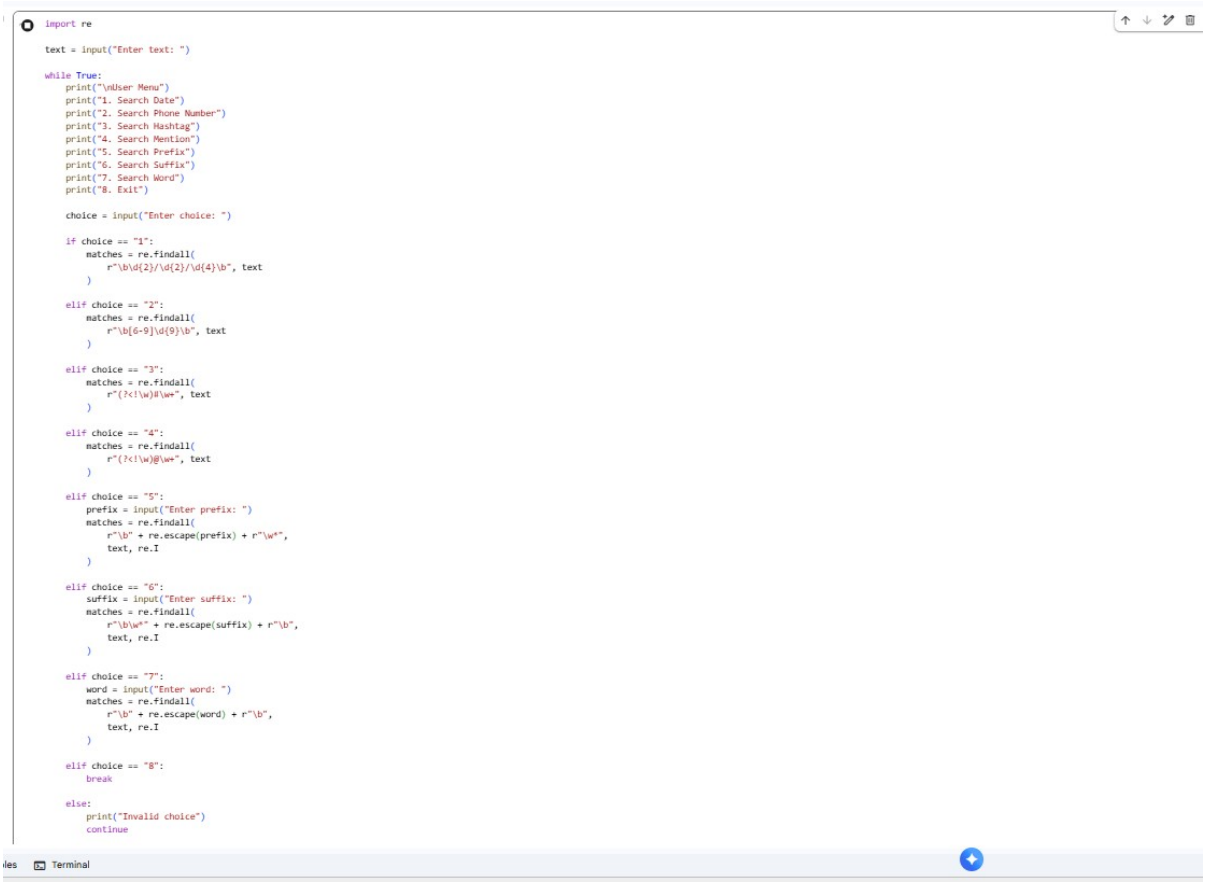
```
        lines.append(line)
    return "\n".join(lines)


def show(title, results):
    print(f"\n{title}:")
    if results:
        for r in results:
            print("  ", r)
    else:
        print("   No match found")


def main():
    text = read_text()
    while True:
        print("\n1.Search Date\n2.Search Phone Number\n3.Search Hashtag\n"
              "4.Search Mention\n5.Search Prefix\n6.Search Suffix\n"
              "7.Search Word\n8.Exit")
        ch = input("Enter choice: ").strip()
        if ch == "1":
            show("Dates found", search(PATTERNS["date"], text))
        elif ch == "2":
            show("Phone numbers found", search(PATTERNS["phone"], text))
        elif ch == "3":
            show("Hashtags found", search(PATTERNS["hashtag"], text))
        elif ch == "4":
            show("Mentions found", search(PATTERNS["mention"], text))
        elif ch == "5":
            p = input("Enter prefix: ").strip()
            show(f"Words starting with '{p}'", prefix_search(p, text))
        elif ch == "6":
            s = input("Enter suffix: ").strip()
            show(f"Words ending with '{s}'", suffix_search(s, text))
        elif ch == "7":
            w = input("Enter word: ").strip()
            show(f"Matches for '{w}'", word_search(w, text))
        elif ch == "8":
            break
        else:
            print("Invalid choice")


if __name__ == "__main__":
    main()
```

3.3 Output



```
import re

text = input("Enter text: ")

while True:
    print("\nUser Menu")
    print("1. Search Date")
    print("2. Search Phone Number")
    print("3. Search Hashtag")
    print("4. Search Mention")
    print("5. Search Prefix")
    print("6. Search Suffix")
    print("7. Search Word")
    print("8. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        matches = re.findall(
            r"\b\d{2}/\d{2}/\d{4}\b", text
        )

    elif choice == "2":
        matches = re.findall(
            r"\b[6-9]\d{9}\b", text
        )

    elif choice == "3":
        matches = re.findall(
            r"(?<!\w)#\w+", text
        )

    elif choice == "4":
        matches = re.findall(
            r"(?<!\w)@\w+", text
        )

    elif choice == "5":
        prefix = input("Enter prefix: ")
        matches = re.findall(
            r"\b" + re.escape(prefix) + r"\w*",
            text, re.I
        )

    elif choice == "6":
        suffix = input("Enter suffix: ")
        matches = re.findall(
            r"\b\w*" + re.escape(suffix) + r"\b",
            text, re.I
        )

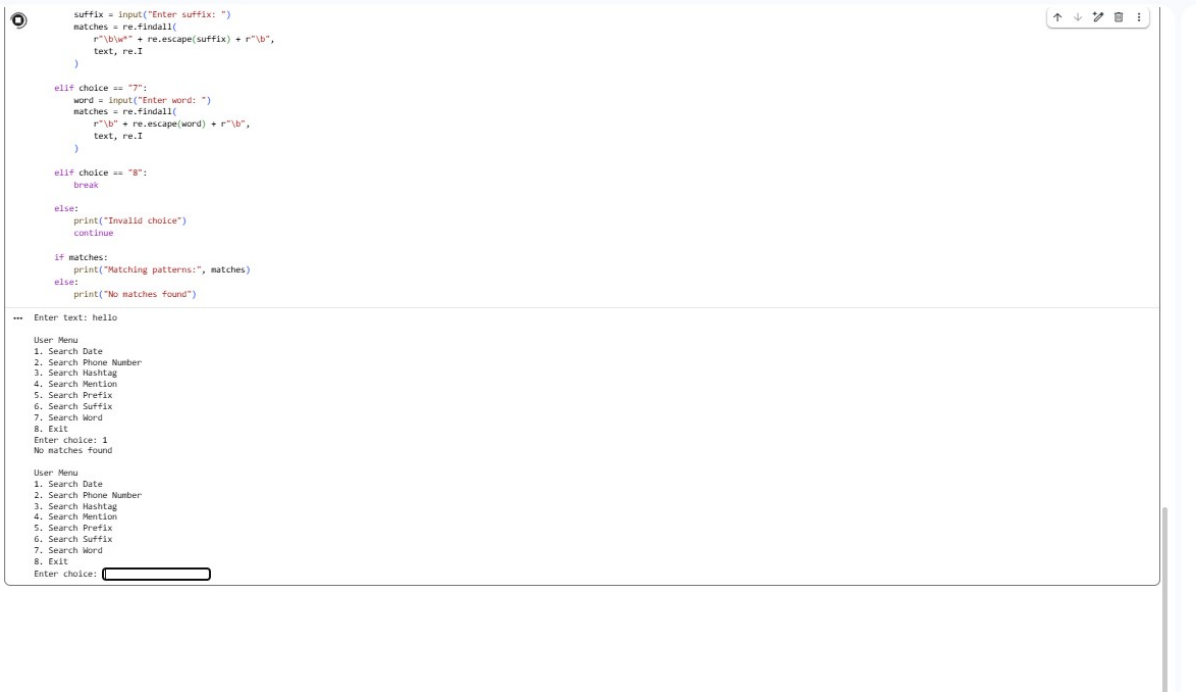
    elif choice == "7":
        word = input("Enter word: ")
        matches = re.findall(
            r"\b" + re.escape(word) + r"\b",
            text, re.I
        )

    elif choice == "8":
        break

    else:
        print("Invalid choice")
        continue
```

Terminal

Figure 3: Pattern matching engine - source code and menu



```
        suffix = input("Enter suffix: ")
        matches = re.findall(
            r"\b\w*" + re.escape(suffix) + r"\b",
            text, re.I
        )

    elif choice == "7":
        word = input("Enter word: ")
        matches = re.findall(
            r"\b" + re.escape(word) + r"\b",
            text, re.I
        )

    elif choice == "8":
        break

    else:
        print("Invalid choice")
        continue

    if matches:
        print("Matching patterns:", matches)
    else:
        print("No matches found")
```

```
Enter text: hello

User Menu
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter choice: 1
No matches found

User Menu
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter choice:
```

Figure 4: Pattern matching engine - menu output

Conclusion

- Regular expressions with lookaheads make multi-rule validation compact and readable.
- A dictionary-based transition table keeps the DFA simulator generic: any DFA can be entered at run time.
- Anchors, character classes and word boundaries allow precise pattern search over free text.